



第5章 结构化实现

主要内容

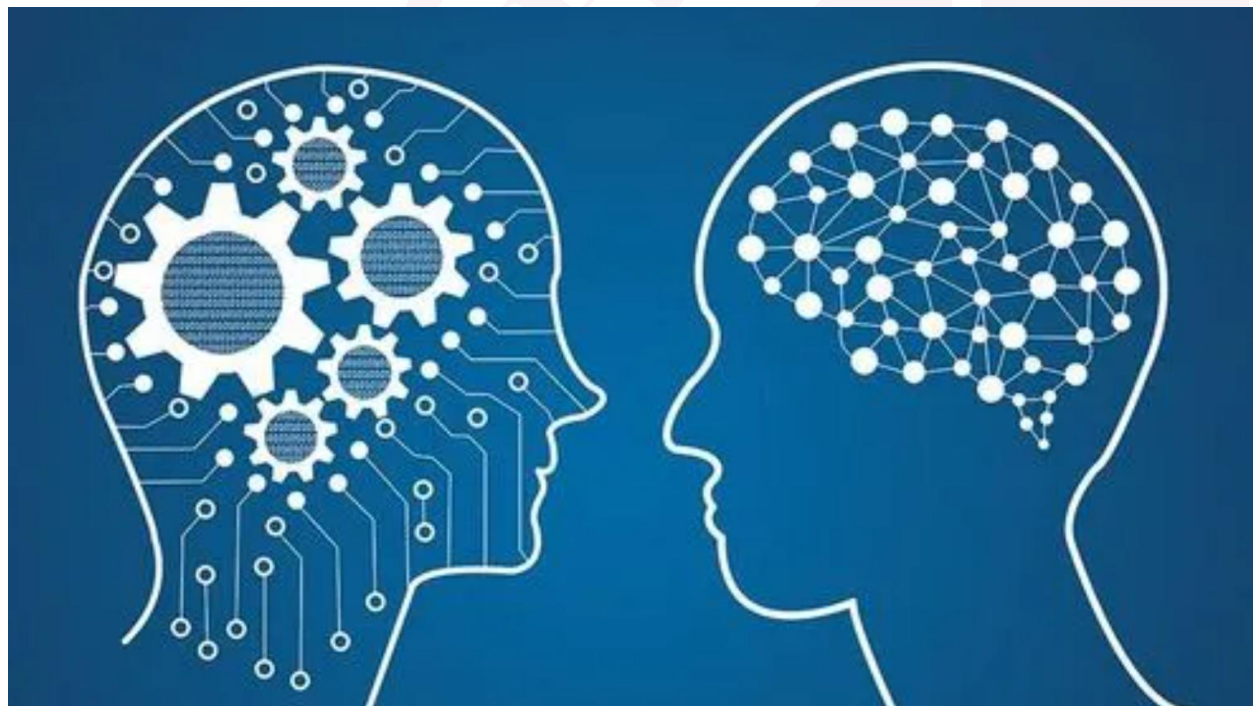
结构化实现与结构化设计的关系

结构化编码

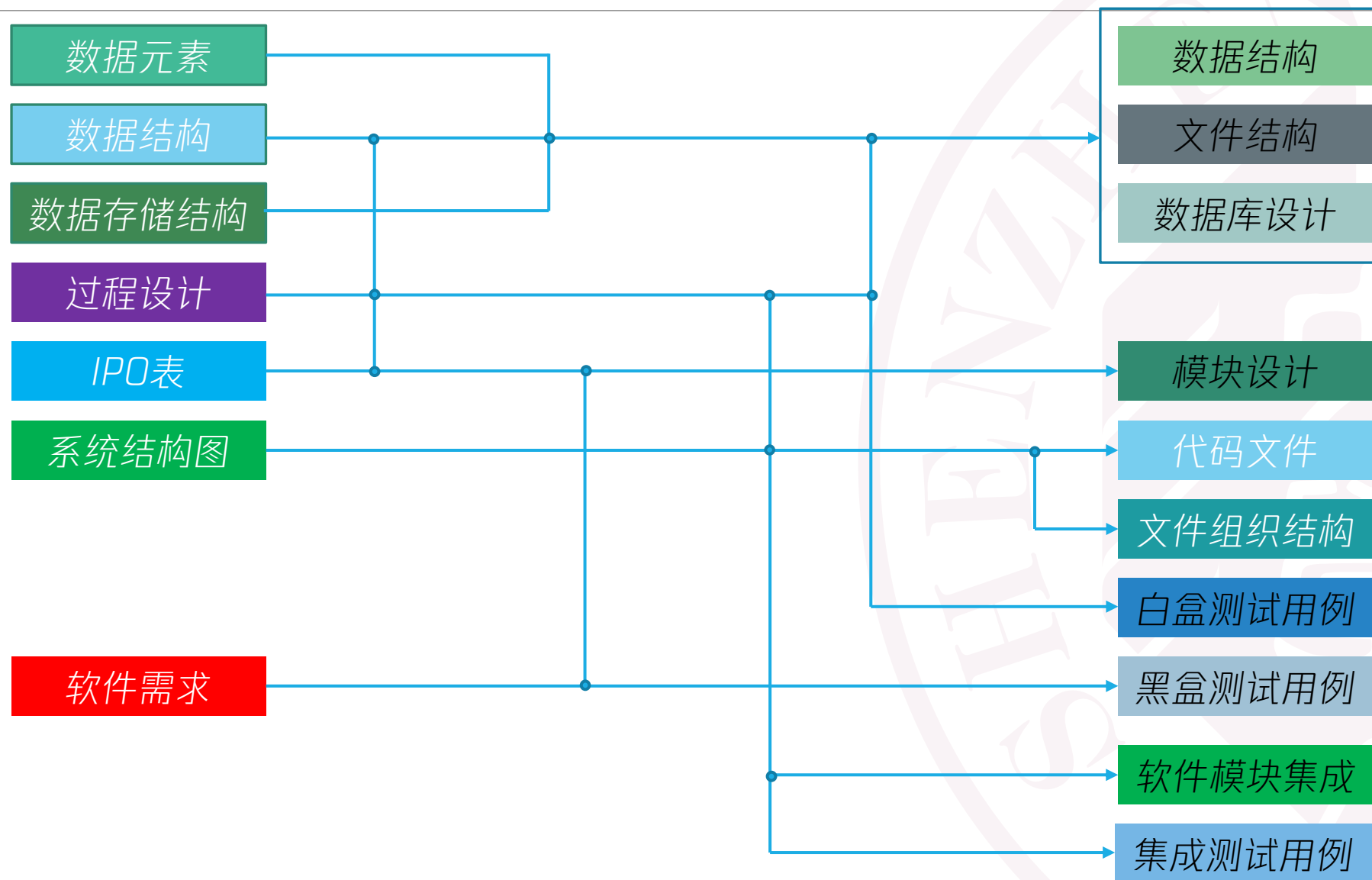
- 结构化程序的源代码组成
- 结构化程序的编译过程
- 结构化程序多文件组织
- 结构化模块集成

结构化测试

- 结构化测试阶段
- 结构化白盒测试覆盖标准
- 结构化白盒测试用例设计
- 结构化黑盒测试用例设计
- 结构化测试实施



1. 结构化实现与结构化设计的关系



2. 结构化编码

[1] 结构化程序的源代码组成

高级程序设计语言均提供了分文件代码组织方式，允许程序设计人员将软件代码分解到多个不同类型的文件中，对代码内容进行分类组织、管理。

C语言编写的项目源代码可以分为：



头文件.h



源程序文件.c



主程序源文件.c

2. 结构化编码

在C语言中的文件主要分为头文件和源程序文件两种。

➤ 头文件

主要用于全局变量、全局宏、函数的声明，以及被其它源代码文件引用共享。

➤ 源程序文件

主要用于实现程序代码逻辑，包括主函数代码实现，以及头文件中定义的业务函数代码实现。

2. 结构化编码

将程序代码分为头文件和源程序文件的优点：

- ✓ 将函数实现写入源程序文件，可以减少中间文件冗余；
- ✓ 将公共变量写入程序头文件中，有利于变量共享；
- ✓ 将宏和数据类型定义写入头文件中，有利于统一数据信息；
- ✓ 将程序业务逻辑写到源程序文件中，有利于保护源代码的内容；
- ✓ 使用头文件可以加强类型安全检查。

2. 结构化编码

[2] 结构化程序的编译过程



2. 结构化编码

① 预处理

编译器首先扫描所有源程序文件，找出包含main函数的源程序文件，即主程序源文件。

将逐行扫描主程序源文件，处理该文件中的文件包含、宏定义、变量声明、函数声明和程序逻辑，并生成一个临时的“C文件”。

② 词法与语法分析

根据程序语法对生成的临时中间文件进行语法检查，确保临时文件中没有语法错误。

2. 结构化编码

③ 编译

先将临时中间文件编译成汇编语句，然后再将汇编语句编译成与CPU相关的二进制代码，生成各个目标文件（.obj文件）。

④ 连接

编译器根据程序设计人员设置的参数信息，将编译阶段产生的多个目标文件与系统库文件进行连接。

编译器对文件中的代码进行绝对地址定位，生成与特定系统平台相关的可执行文件。

2. 结构化编码

[3] 结构化程序多文件组织

合理组织代码内容，将关系比较紧密的多个函数放入同一个源程序文件中，将关联比较紧密的函数定义、变量定义、宏定义等内容放入同一个头文件中。

程序设计人员也可以根据“系统结构图”中的模块划分来组织源程序文件中的内容。

2. 结构化编码

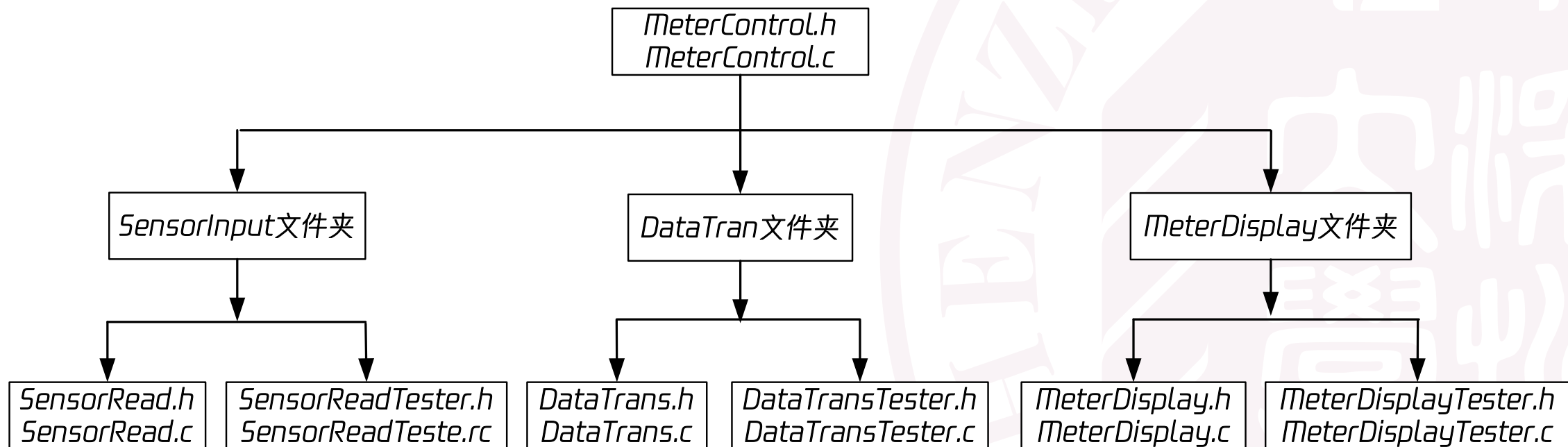
划分程序内容步骤:

- ✓ ① 创建主程序源文件
- ✓ ② 为系统结构图中的模块定义头文件和源程序文件
- ✓ ③ 为系统结构图中的各个模块定义对应的测试文件

如果程序的规模较大，程序设计人员可以结合“系统结构图”中的模块划分内容，利用文件夹来组织项目文件。

2. 结构化编码

汽车仪表盘项目文件组织案例:



2. 结构化编码

[4] 结构化模块集成

根据程序开发人员集成软件模块的方式不同:

- ✓ 非渐增式组装: 所有的软件模块同时集成;
- ✓ 渐增式组装: 按照指定的顺序, 逐一集成各个软件模块。
 - 自顶向下集成
 - 自底向上集成

2. 结构化编码

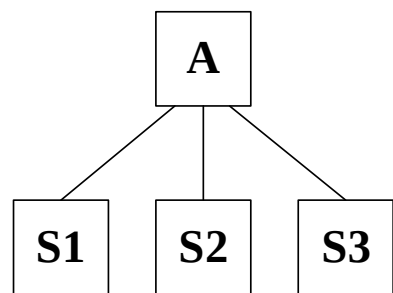
在模块集成过程中，软件开发团队可以采用桩模块替代下层模块。

所谓桩模块，也可以称为存根模块，主要用于下层模块未完成或者无法获得情况下的软件集成。

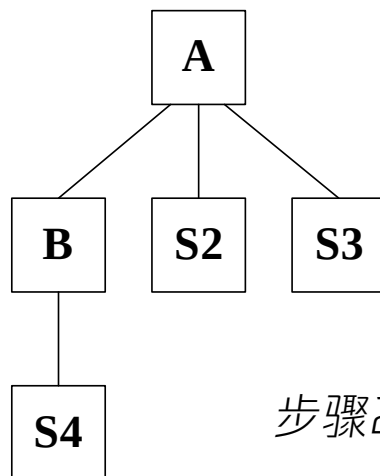
在软件测试和集成过程中，桩模块可以作为下层模块，以及被调模块的替代模块。

2. 结构化编码

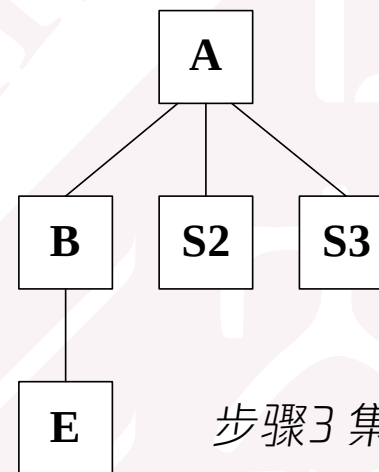
① 自顶向下集成



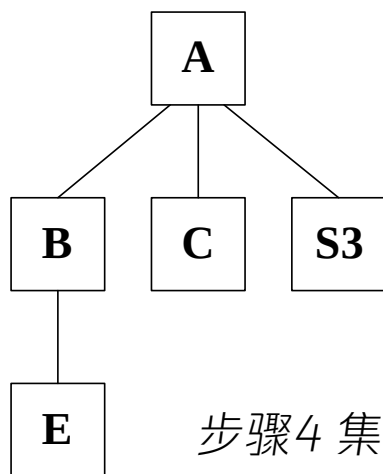
步骤1



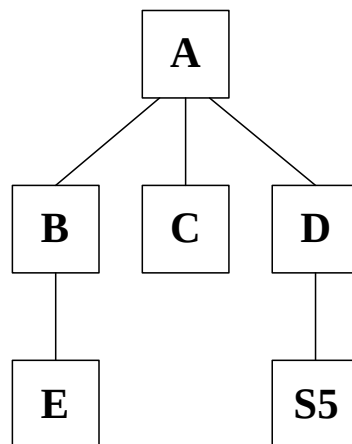
步骤2 集成模块B



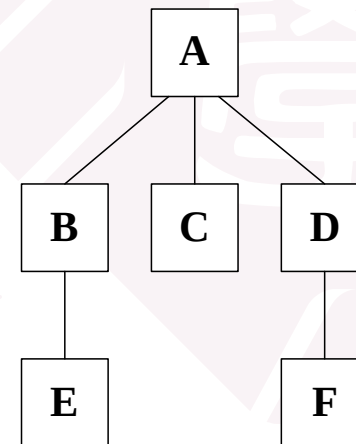
步骤3 集成模块E



步骤4 集成模块C



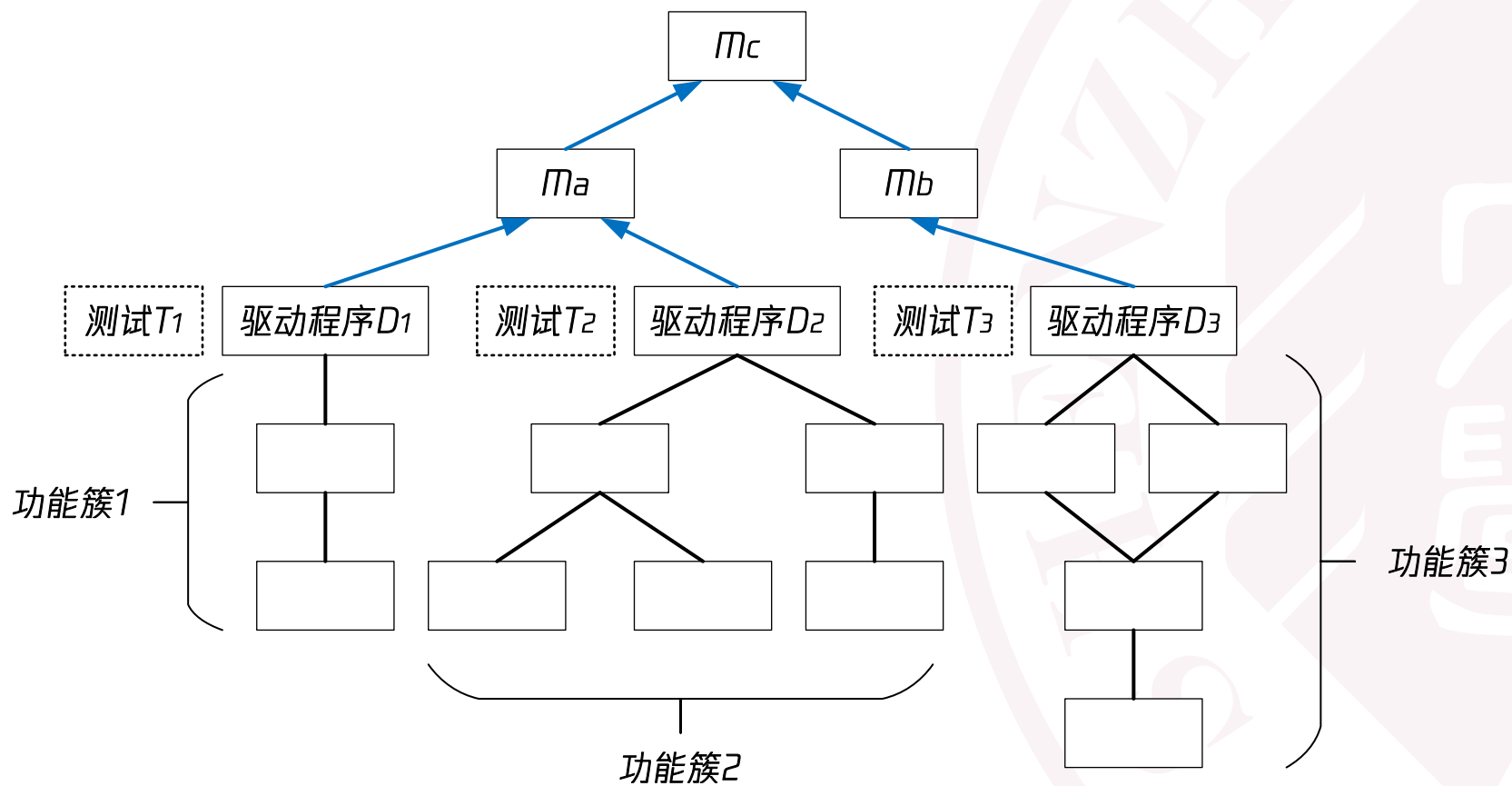
步骤5 集成模块D



步骤6 集成模块F

2. 结构化编码

② 自底向上集成



2. 结构化编码

自顶向下集成

优点：直观，无需编写集成、测试驱动程序

能够较早发现上层模块在控制方面存在的问题

缺点：需要编写大量桩模块

不容易发现底层模块中存在的问题

自底向上集成

优点：容易发现底层模块中出现的问题

多个功能簇可以并行集成和测试

缺点：主控模块的控制问题要到最后才能发现和解决

2. 结构化编码

③ 改进的自顶向下集成

基本上使用自顶向下集成

在项目集成初期，先使用自底向上集成方式进行集成，减少自顶向下集成方式中撰写桩模块的工作量。

3. 结构化测试

[1] 结构化测试阶段

结构化分析	结构化设计	结构化实现	最终软件
· 数据处理过程 · 数据处理逻辑 · 系统状态 · 数据模型 · 数据字典	· 系统结构图 · IPO表 · 数据设计 · 过程设计	· 编码风格 · 代码与设计是否一致 · 系统是否正确集成	· 功能是否满足需求 · 性能是否满足需求等

结构化测试各阶段的测试内容

3. 结构化测试

[2] 结构化白盒测试覆盖标准

白盒测试是指根据模块的内部逻辑来设计测试用例。



3. 结构化测试

① 语句覆盖

确保被测模块中的每条语句至少执行一次。

```
if(A && (B || C))  
    x = 1;  
else  
    x = 0;
```

	测试用例 1	测试用例 2
A	T	F
B	T	F
C	T	F
A && (B C)	T	F
A (B C)	T	F
语句覆盖率	100%	

无法判别!

3. 结构化测试

② 判定〔分支〕覆盖

保证被测模块中的每个判定的分支至少执行一次。

	1	2	3	4	5	6	7	8
A	F	F	F	F	T	T	T	T
B	F	F	T	T	F	F	T	T
C	F	T	F	T	F	T	F	T
A && (B C)	F	F	F	F	F	T	T	T
A && (B && C)	F	F	F	F	F	F	F	T
判定覆盖率	50%							50%

```
if(A && (B || C))  
    x = 1;  
else  
    x = 0;
```

无法判别!

3. 结构化测试

③ 条件覆盖

确保被测模块中的每个条件都至少为真一次，为假一次。

	测试用例 1	测试用例 2
A	F	T
B	T	F
C	T	F
A && (B C)	F	F
条件覆盖率	100%	
判定覆盖率	50%	

```
if(A && (B || C))  
    x = 1;  
else  
    x = 0;
```

条件覆盖不一定能够达到完全的判定覆盖。

3. 结构化测试

④ 判定/条件覆盖

确保被测模块同时满足判定覆盖和条件覆盖两种标准。

	1	2	3	4	5	6	7	8
A	F	F	F	F	T	T	T	T
B	F	F	T	T	F	F	T	T
C	F	T	F	T	F	T	F	T
A && (B C)	F	F	F	F	F	T	T	T
A && (B && C)	F	F	F	F	F	F	F	T

```
if(A && (B || C))  
    x = 1;  
else  
    x = 0;
```

无法判别!

3. 结构化测试

⑤ 条件组合覆盖

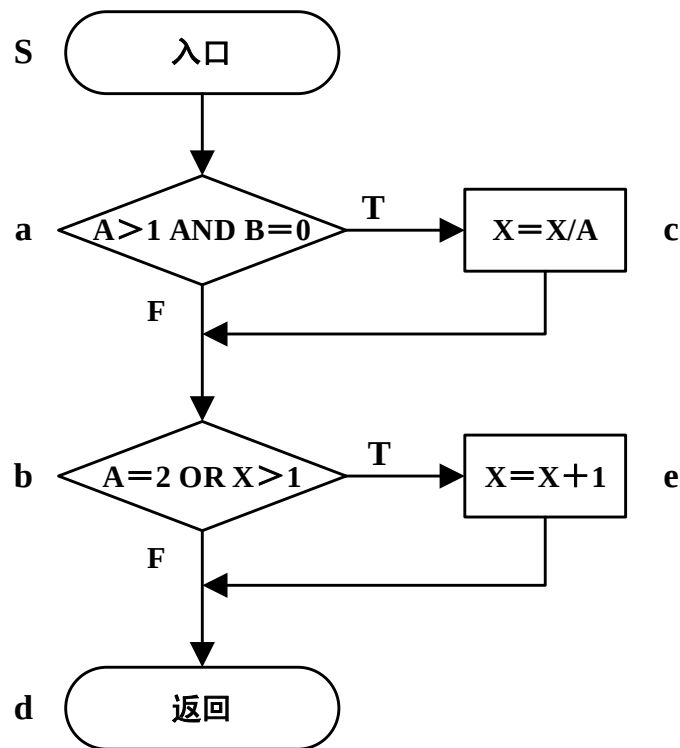
确保被测模块中所有条件的每一种组合至少为真一次，为假一次。

	1	2	3	4	5	6	7	8
A	F	F	F	F	T	T	T	T
B	F	F	T	T	F	F	T	T
C	F	T	F	T	F	T	F	T
B C	F	T	T	T	F	T	T	T
A && (B C)	F	F	F	F	F	T	T	T

```
if(A && (B || C))  
    x = 1;  
else  
    x = 0;
```

3. 结构化测试

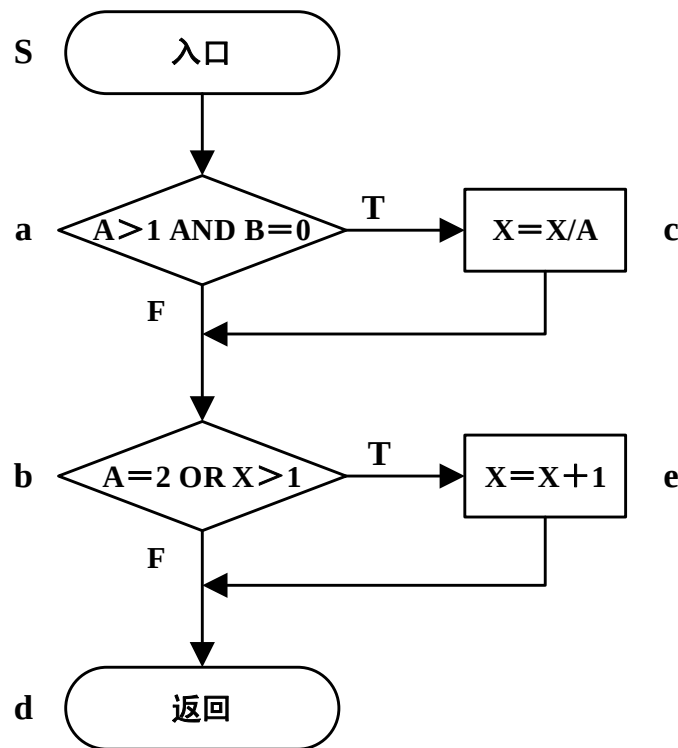
白盒测试案例



关键变量: A 、 B 、 X
取值组合: $[A, B, X]$

3. 结构化测试

白盒测试案例



语句覆盖:

测试用例[2, 0, 4] 覆盖语句sacbed

判定覆盖:

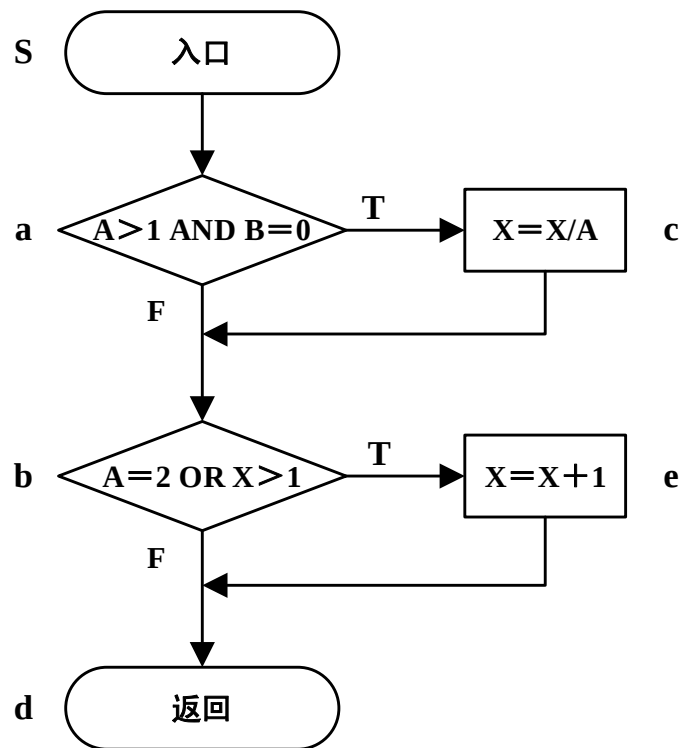
- 测试用例[3, 0, 1] 覆盖语句sacbd, 让判定1为真一次, 判定2为假一次;
- 测试用例[2, 1, 1]覆盖语句sabed, 让判定1为假一次, 判定2为真一次。

条件覆盖:

- 测试用例[3, 0, 1]覆盖语句sacbd, 让条件1为真一次, 条件2为真一次, 条件3为假一次, 条件4为假一次;
- 测试用例[1, 1, 2]覆盖语句sabed, 让条件1为假一次, 条件2为假一次, 条件3为假一次, 条件4为真一次;
- 测试用例[2, 1, 2]覆盖语句sabed, 让条件1为假一次, 条件2为假一次, 条件3为真一次, 条件4为真一次;

3. 结构化测试

白盒测试案例

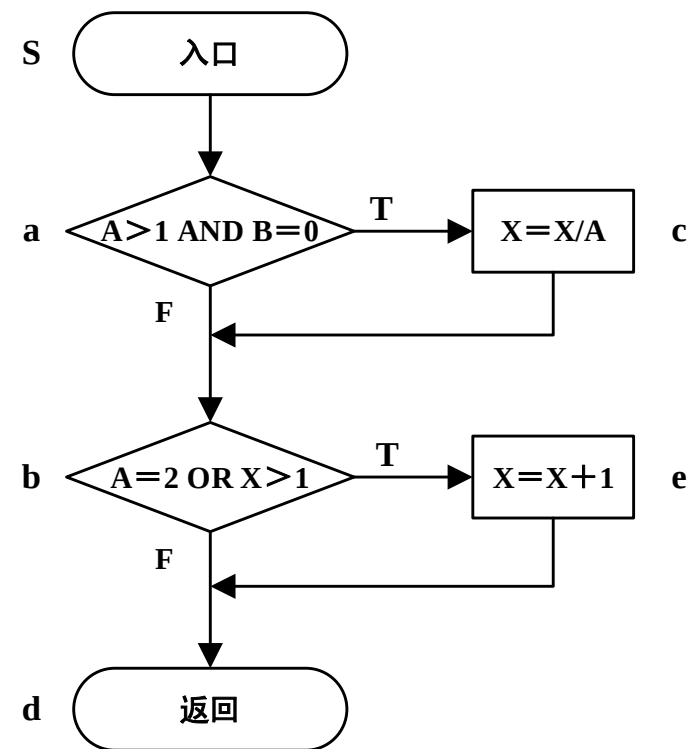


判定条件覆盖:

- 测试用例[3, 0, 1]覆盖语句sacbd, 让判定1为真一次, 判定2为假一次, 条件1为真一次, 条件2为真一次, 条件3为假一次, 条件4为假一次;
- 测试用例[1, 1, 2]覆盖语句sabcd, 让判定1为假一次, 判定2为真一次, 条件1为假一次, 条件2为假一次, 条件3为假一次, 条件4为真一次;
- 测试用例[2, 1, 2]覆盖语句sabcd, 让判定1为假一次, 判定2为真一次, 条件1为假一次, 条件2为假一次, 条件3为真一次, 条件4为真一次。

3. 结构化测试

白盒测试案例



令：条件“A>1”为T1
条件“B=0”为T2
条件“A=2”为T3
条件“X>1”为T4

序号	条件组合	效果
1	A>1,B=0	T1,T2
2	A>1,B!=0	T1,-T2
3	A<=1,B=0	-T1,T2
4	A<=1,B!=0	-T1,-T2
5	A=2,X>1	T3,T4
6	A=2,X<=1	T3,-T4
7	A!=2,X>1	-T3,T4
8	A!=2,X<=1	-T3,-T4

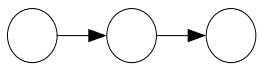
条件组合覆盖:

序号	输入[A,B,X]	覆盖组合	覆盖条件	执行路径
1	2 0 4	1, 5	T1,T2,T3,T4	T T
2	2 1 1	2, 6	T1,-T2,T3,-T4	F T
3	1 0 2	3, 7	-T1,T2,-T3,T4	F T
4	1 1 1	4, 8	-T1,-T2,-T3,-T4	F F

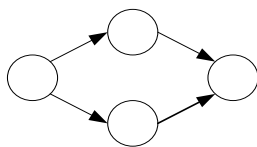
3. 结构化测试

[3] 结构化白盒测试用例设计

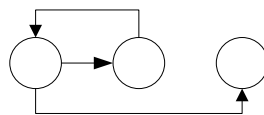
程序控制流图，简称流图，对程序的控制流程进行建模。



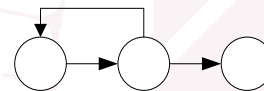
顺序流程



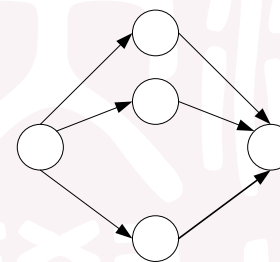
选择流程



while循环流程



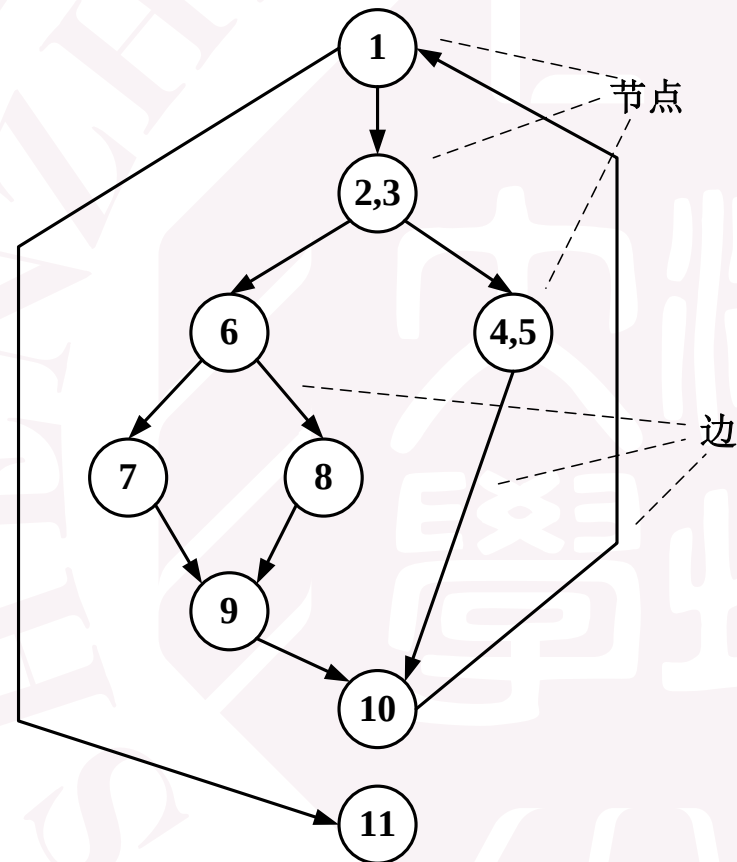
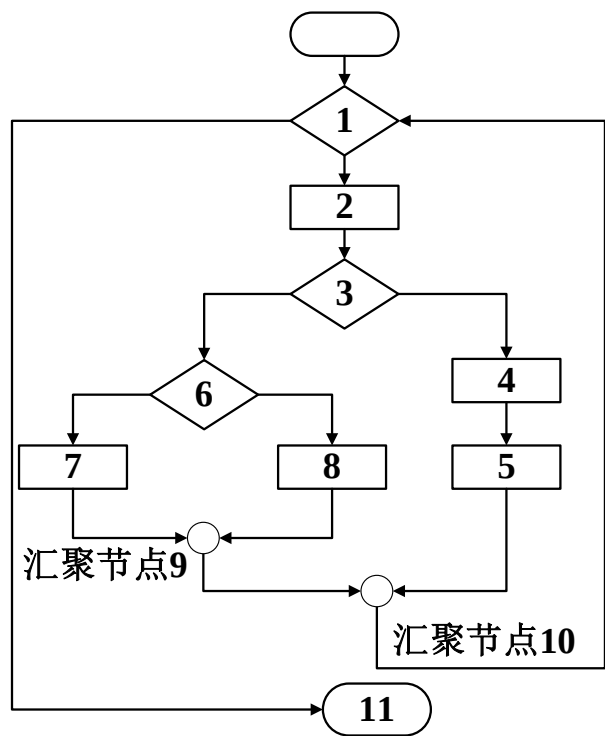
until循环流程



多分支流程

3. 结构化测试

程序控制流图转换案例



3. 结构化测试

复合表达式分解案例

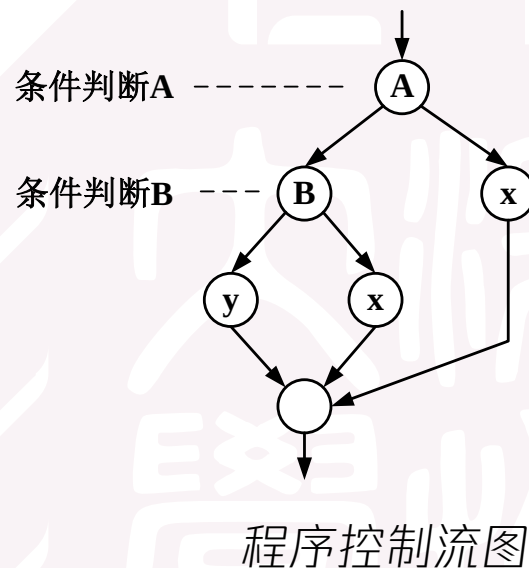
```
if A OR B
  then procedure x;
else
  procedure y;
```

原始判定



```
if A
  then procedure x;
else
  if B
    then procedure x;
  else
    procedure y;
```

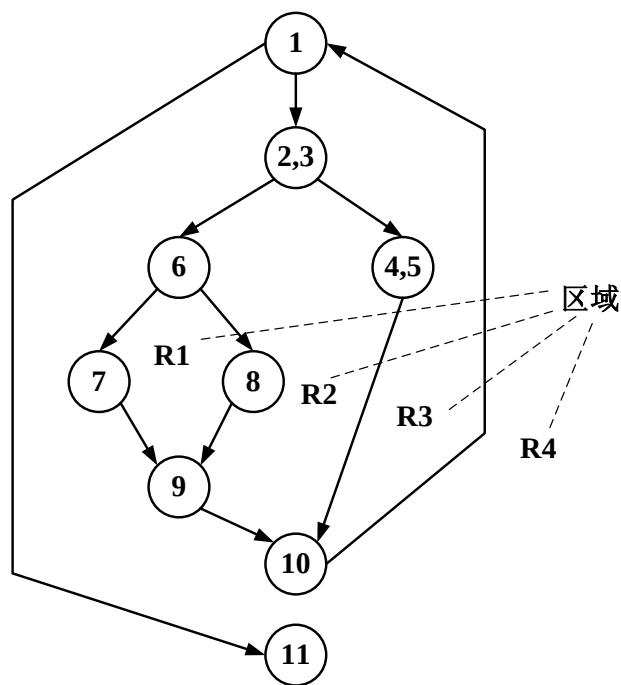
分解后的判定



3. 结构化测试

控制流图环形复杂度 $V[G]$ 分析

[1] 等于流图中的区域数

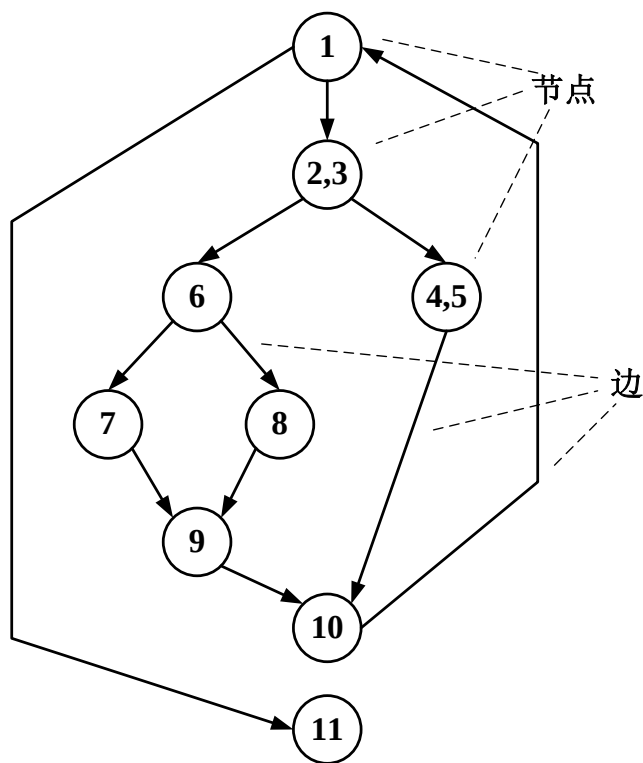


区域数: 4
 $V[G] = 4$

3. 结构化测试

控制流图环形复杂度 $V[G]$ 分析

[2] 等于图中边的数量减去流图中的节点数, 然后再加2



边数 E : 11

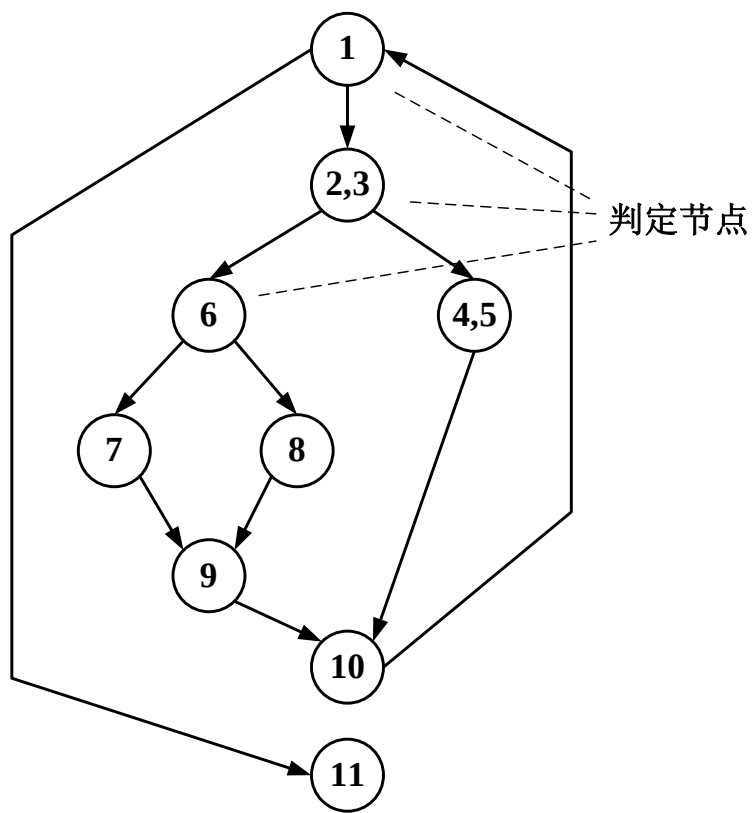
节点数 N : 9

$$V[G] = E - N + 2 = 4$$

3. 结构化测试

控制流图环形复杂度 $V[G]$ 分析

[3] 等于流图中判定节点的数量 + 1



判定节点数 P : 3
 $V[G] = P + 1 = 4$

3. 结构化测试

路径覆盖

确保模块程序控制流图中的所有路径被至少遍历一次。

路径覆盖可以等同于条件组合覆盖。

独立路径：至少引入一个新处理语句集合或一个新条件的路径

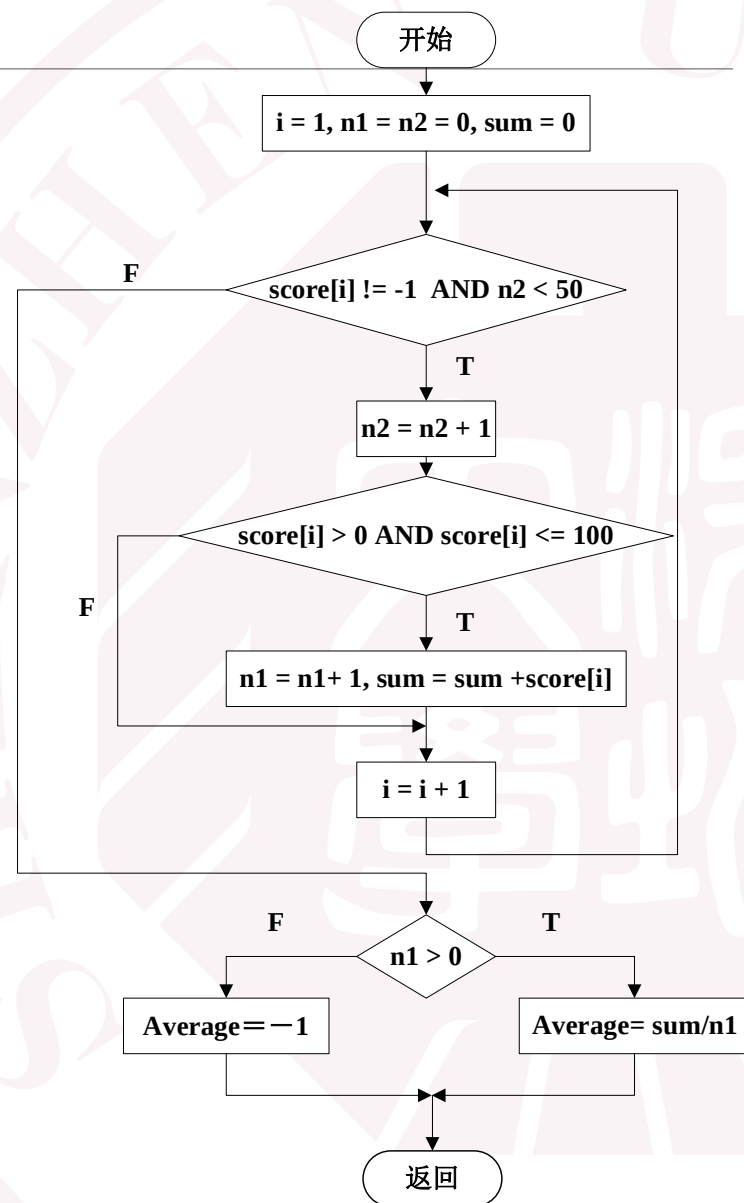
独立路径的数量 $\leq$ 流图的环形复杂度

3. 结构化测试

案例名称：统计、求解学生成绩

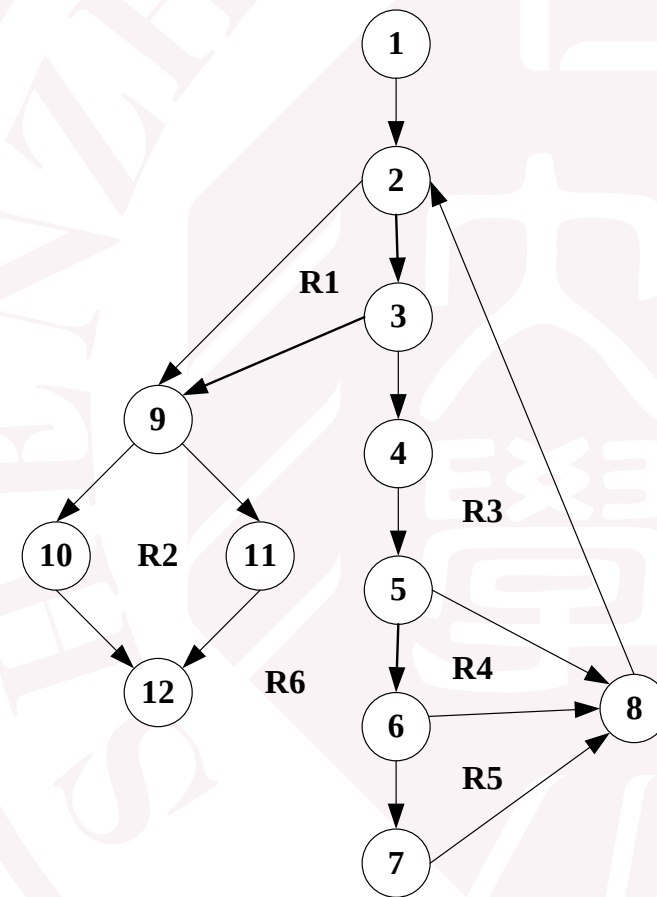
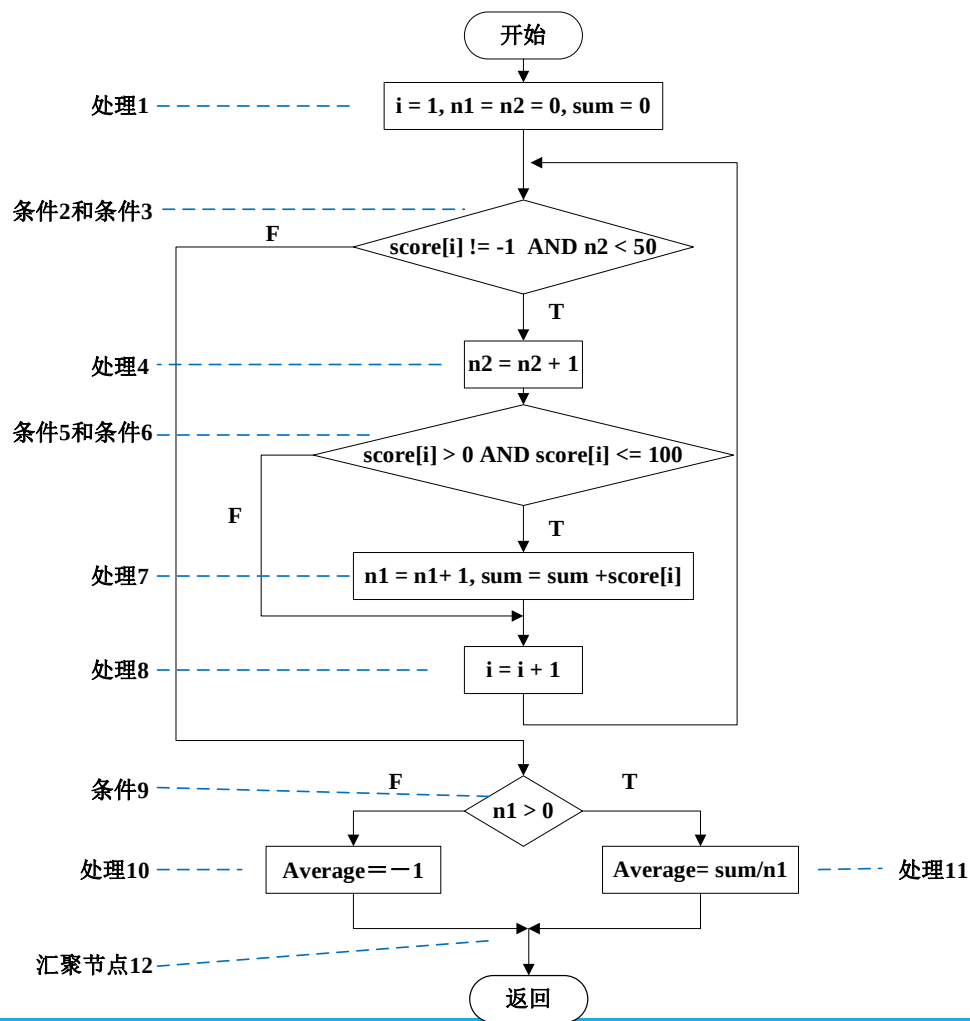
某个数组中最多可以存储50个同学的成绩。

其中，成绩 - 1为输入结束标志。



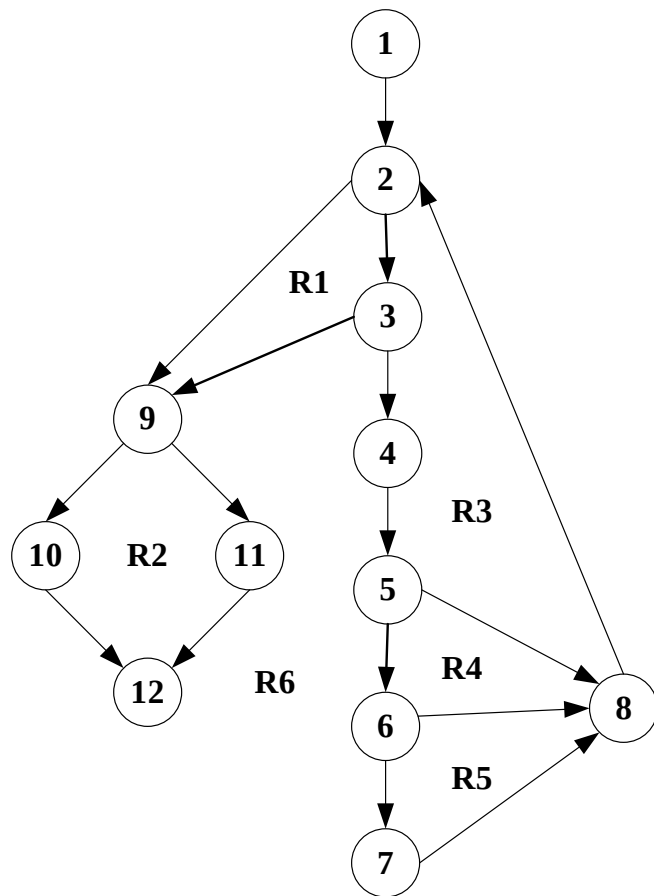
3. 结构化测试

步骤1: 将模块的内部处理流程转换为程序控制流图



3. 结构化测试

步骤2: 分析程序控制流图的环形复杂度, 确定程序控制流图中独立路径的数量



方法1:

区域数: 6

环形复杂度 $V[G] = 6$

方法2:

边数 E : 16

节点数 n : 12

$V[G] = E - n + 2 = 6$

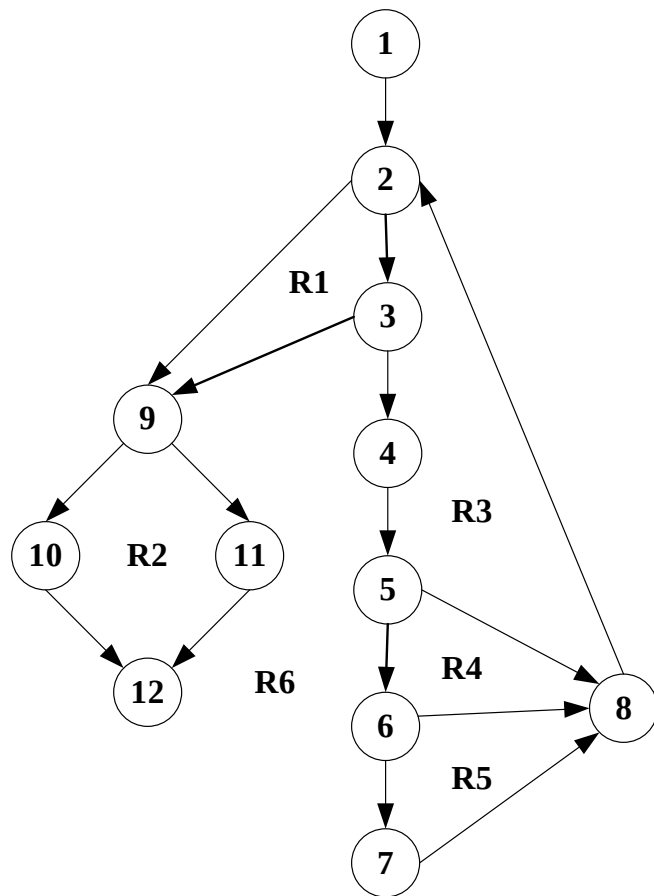
方法3:

判定节点数 P : 5

$V[G] = P + 1 = 6$

3. 结构化测试

步骤3: 根据程序控制流图和程序控制流图的环形复杂度确定程序基本路径集合中的独立路径;

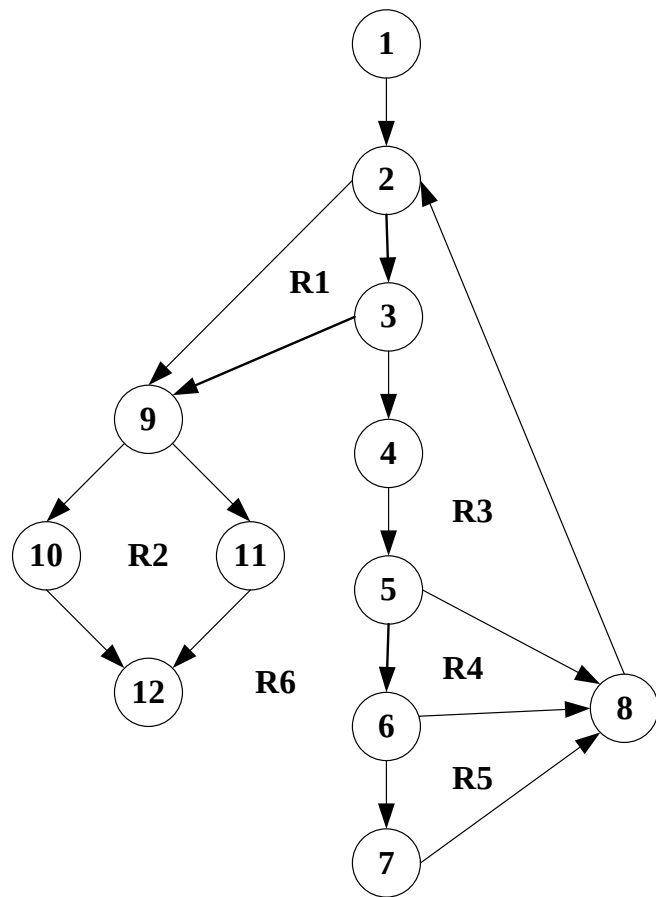


独立路径集合:

- 路径1: 1-2-9-10-12
- 路径2: 1-2-9-11-12
- 路径3: 1-2-3-9-10-12
- 路径4: 1-2-3-4-5-8-2...
- 路径5: 1-2-3-4-5-6-8-2...
- 路径6: 1-2-3-4-5-6-7-8-2...

3. 结构化测试

步骤4: 为每一条独立路径设计一组测试用例, 确保每条独立路径至少被执行一次



路 径 1: 1-2-9-10-12

输 入: $score[1] = -1$

期望结果: $average = -1$, 其他量保持初值。

路 径 2: 1-2-9-11-12

输 入: $score[k] = \text{有效分数值}$, 且 $k < i$
 $score[i] = -1, 2 \leq i \leq 50$;

期望结果: 根据输入的有效分数算出正确的分数个数 $n1$ 、总分 sum 和平均分 $average$ 。

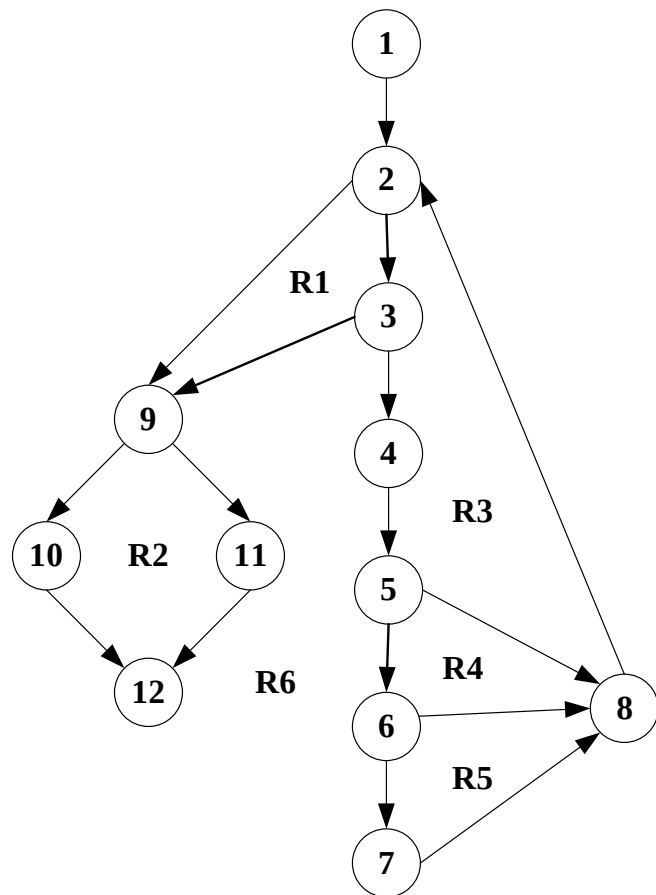
路 径 3: 1-2-3-9-10-12

输 入: 输入的有效成绩多于50个, 即试图处理51个分数;

期望结果: $n1 = 50$, 且算出正确的总分和平均分。

3. 结构化测试

步骤4: 为每一条独立路径设计一组测试用例, 确保每条独立路径至少被执行一次



路 径 4: 1-2-3-4-5-8-2...

输 入: $score[i] < 0$, 且 $i < 50$;

期望结果: 根据输入的有效分数算出正确的分数个数 $n1$ 、总分 sum 和平均分 $average$ 。

路 径 5: 1-2-3-4-5-6-8-2...

输 入: $score[i] > 100$, 且 $i < 50$;

期望结果: 根据输入的有效分数算出正确的分数个数 $n1$ 、总分 sum 和平均分 $average$ 。

路 径 6: 1-2-3-4-5-6-7-8-2...

输 入: $score[i] = \text{有效分数}$, 且 $i < 50$;

期望结果: 根据输入的有效分数算出正确的分数个数 $n1$ 、总分 sum 和平均分 $average$ 。

3. 结构化测试

其他测试用例设计方法

- 条件测试法
- 数据流测试法
- 循环测试法

3. 结构化测试

[4] 结构化黑盒测试用例设计

根据模块的规格说明来设计或者选择测试用例，测试模块实现的功能是否与需求规格说明书相符。

3. 结构化测试

黑盒测试案例

模块名：判断三角形形状

输入：三条边长

输出：能否组成三角形

能否组成等腰或等边三角形



判断标准：

- [1] $A > 0$, $B > 0$, $C > 0$, 且 $A + B > C$, $B + C > A$, $A + C > B$, 能够构成三角形;
- [2] 如果三条边能够组成三角形, 且当 $A = B$, 或 $B = C$, 或 $A = C$ 时, 能够组成等腰三角形;
- [3] 如果三条边能够组成三角形, 且当 $A = B$, 且 $B = C$, 且 $A = C$ 时, 能够组成等边三角形。

3. 结构化测试

[1] 等价类划分法

输入条件	有效等价类		无效等价类	
三角形的 3 条边	$(A > 0)$	(1)	$(A \leq 0)$	(7)
	$(B > 0)$	(2)	$(B \leq 0)$	(8)
	$(C > 0)$	(3)	$(C \leq 0)$	(9)
	$(A + B > C)$	(4)	$(A + B \leq C)$	(10)
	$(B + C > A)$	(5)	$(B + C \leq A)$	(11)
	$(A + C > B)$	(6)	$(A + C \leq B)$	(12)
是否等腰三角形	$(A = B)$	(13)	$(A \neq B) \text{ and } (B \neq C) \text{ and } (C \neq A)$	(16)
	$(B = C)$	(14)		
	$(C = A)$	(15)		
是否等边三角形	$(A = B) \text{ and } (B = C) \text{ and } (C = A)$	(17)	$(A \neq B)$ $(B \neq C)$ $(C \neq A)$	(18) (19) (20)

3. 结构化测试

序号	(A, B, C)	覆盖等价类	输出
1	(3, 4, 5)	(1), (2), (3), (4), (5), (6)	一般三角形
2	(0, 1, 2)	(7)	不能构成三角形
3	(1, 0, 2)	(8)	
4	(1, 2, 0)	(9)	
5	(1, 2, 3)	(10)	
6	(1, 3, 2)	(11)	
7	(3, 1, 2)	(12)	
8	(3, 3, 4)	(1), (2), (3), (4), (5), (6), (13)	等腰三角形
9	(3, 4, 4)	(1), (2), (3), (4), (5), (6), (14)	
10	(3, 4, 3)	(1), (2), (3), (4), (5), (6), (15)	
11	(3, 4, 5)	(1), (2), (3), (4), (5), (6), (16)	非等腰三角形
12	(3, 3, 3)	(1), (2), (3), (4), (5), (6), (17)	是等边三角形
13	(3, 4, 4)	(1), (2), (3), (4), (5), (6), (14), (18)	非等边三角形
14	(3, 4, 3)	(1), (2), (3), (4), (5), (6), (15), (19)	
15	(3, 3, 4)	(1), (2), (3), (4), (5), (6), (13), (20)	

① 设计有效等价类测试用例

一个好的有效测试用例应尽可能多地覆盖那些尚未被覆盖的有效等价类。

② 设计无效等价类测试用例

一个好的无效测试用例只能覆盖一个尚未被覆盖的无效等价类。

3. 结构化测试

② 边界值分析

模块名：判断三角形形状

输入：三条边长[1, 100]

输出：能否组成三角形

能否组成等腰或等边三角形

测试用例	A	B	C	预期输出
Test1	60	60	1	等腰三角形
Test2	60	60	2	等腰三角形
Test3	60	60	60	等边三角形
Test4	50	50	99	等腰三角形
Test5	50	50	100	非三角形
Test6	60	1	60	等腰三角形
Test7	60	2	60	等腰三角形
Test8	50	99	50	等腰三角形
Test9	50	100	50	非三角形
Test10	1	60	60	等腰三角形
Test11	2	60	60	等腰三角形
Test12	99	50	50	等腰三角形
Test13	100	50	50	非三角形

3. 结构化测试

其他黑盒测试方法

- ✓ 结合因果图法
- ✓ 判定表驱动法
- ✓ 错误推测法

3. 结构化测试

[5] 结构化测试实施



3. 结构化测试

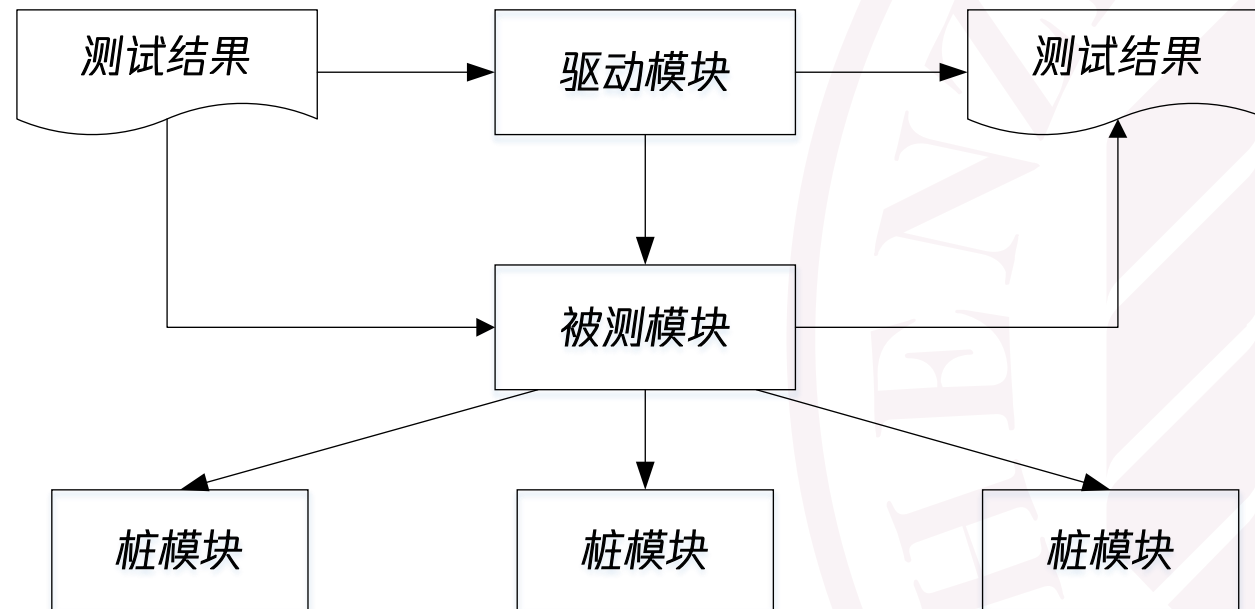
[1] 单元测试

在开展模块测试时，软件测试人员一般需要编写以下两种辅助模块：

驱动模块：用于调用被测模块，根据设计的测试用例，采用不同形式来测试被测模块。

存根模块：根据功能需要直接返回正确值。

3. 结构化测试



3. 结构化测试

[2] 集成测试

根据概要设计阶段得到的系统结构图，按照指定顺序集成各个完成的、经过测试的软件模块。

结构化集成实际上是系统结构图中上层模块内部细节的实现过程。

4. 小结

- 结构化和结构化分析是结构化软件实现的基础;
- 程序源代码分文件管理可以降低软件开发复杂度, 提高软件开发效率;
- 软件开发人员可以通过渐增方式将多个模块组装成为目标软件;
- 软件测试人员通过软件测试排除各个软件开发阶段中存在的错误。
 - ◆ 结构化白盒测试
 - ◆ 结构化黑盒测试

谢谢大家!

课后思考:

1. 如何高效地让多个人合作开发结构化程序?
2. 在开发结构化程序时, 如何保障或随时开展测试?
3. 结构化方法的本质你掌握了吗?